

AutoHub — Sistema de Revenda de Veículos

Relatório Técnico | Renato Ramos | Programação Orientada a Objetos

1. Descrição do sistema

O AutoHub é um sistema desktop em Java para gerenciamento de uma revenda de veículos. Permite cadastrar, buscar, editar e remover veículos do estoque, registrar vendas, retornar veículos ao estoque e simular parcelas de financiamento. A interface gráfica foi desenvolvida com Swing e os dados são persistidos em arquivo binário. O projeto aplica herança, interfaces, exceções próprias, coleção polimórfica e arquitetura em camadas (model, service, persistence, exception, view).

2. Diagrama UML simplificado

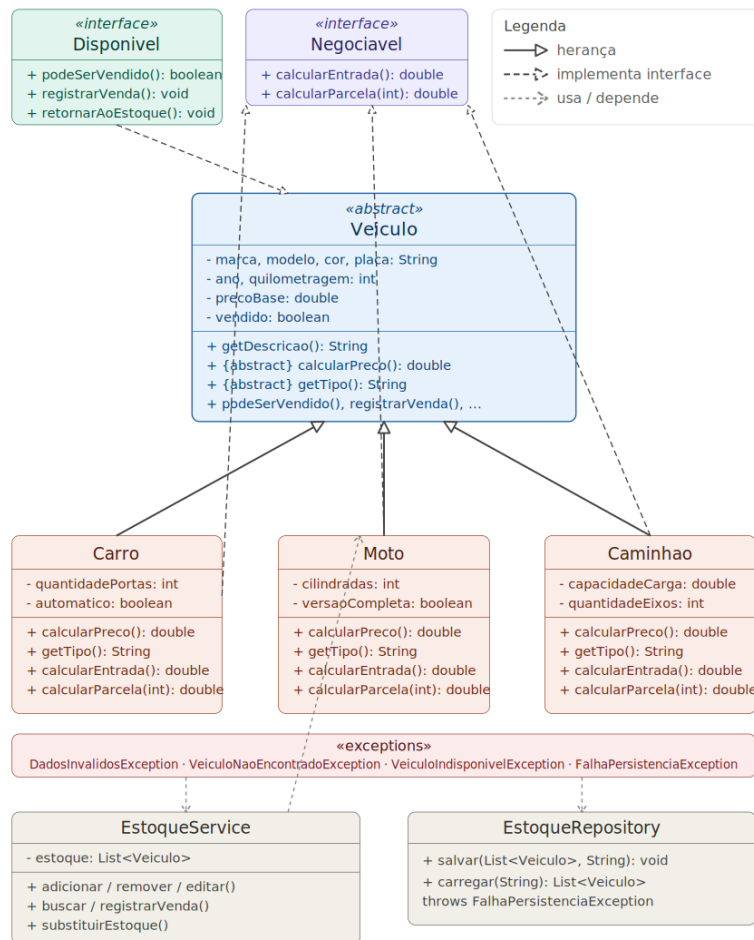


Figura 1 — Diagrama UML do AutoHub

3. Hierarquia de classes

A classe abstrata **Veiculo** centraliza os atributos comuns (marca, modelo, ano, cor, placa, quilometragem, precoBase, vendido) e declara os métodos abstratos **calcularPreco()** e **getTipo()**. As três subclasses concretas herdam de Veiculo e implementam esses métodos com regras próprias:

- **Carro**: acrescenta quantidadePortas e automatico — preço inclui acréscimos por câmbio e portas;
- **Moto**: acrescenta cilindradas e versaoCompleta — preço calculado com base nesses atributos;
- **Caminhao**: acrescenta capacidadeCarga e quantidadeEixos — preço considera ambos.

4. Interfaces utilizadas

Disponível

Implementada pela classe abstrata `Veiculo`. Define `podeSerVendido()`, `registrarVenda()` e `retornarAoEstoque()`, controlando a disponibilidade do veículo no estoque.

```
boolean podeSerVendido(); void registrarVenda(); void retornarAoEstoque();
```

Negociável

Implementada por `Carro`, `Moto` e `Caminhao`. Define `calcularEntrada()` e `calcularParcela(int)`, com percentuais de entrada distintos por tipo (20%, 15% e 30%). Usada polimorficamente na simulação de parcelas via `cast` para `Negociavel`.

```
double calcularEntrada(); double calcularParcela(int quantidadeParcelas);
```

5. Exceções próprias

Exceção	Quando é lançada
<code>DadosInvalidosException</code>	Campos vazios, ano inválido, quilometragem negativa, preço zero ou placa duplicada
<code>VeiculoNaoEncontradoException</code>	Busca, edição ou remoção por palavra-chave sem resultado
<code>VeiculoIndisponivelException</code>	Tentativa de registrar venda de veículo já vendido
<code>FalhaPersistenciaException</code>	Erro de I/O ao salvar ou carregar dados em arquivo

6. Propagação de exceção entre camadas

Exemplo: tentativa de vender um veículo já vendido. A camada **service** aplica a regra e lança a exceção; a camada **view** captura e exibe ao usuário:

```
// EstoqueService.java (service)
if (!veiculo.podeSerVendido())
    throw new VeiculoIndisponivelException("Este veículo já foi vendido.");

// AutoHubFrame.java (view)
catch (...) | VeiculoIndisponivelException erro {
    mostrarErro("Erro ao registrar venda", erro.getMessage());
}
```

7. Coleção polimórfica

O `EstoqueService` mantém um `ArrayList<Veiculo>` que armazena objetos de `Carro`, `Moto` e `Caminhao` na mesma coleção. Ao iterar, o polimorfismo garante que cada objeto execute sua própria implementação de `calcularPreco()` e `getTipo()`. Suporta as operações de adicionar, buscar por palavra-chave, editar e remover.

8. Interface gráfica

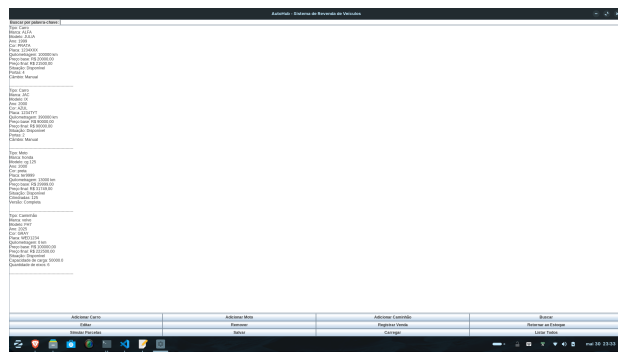


Figura 2 — Tela principal com listagem

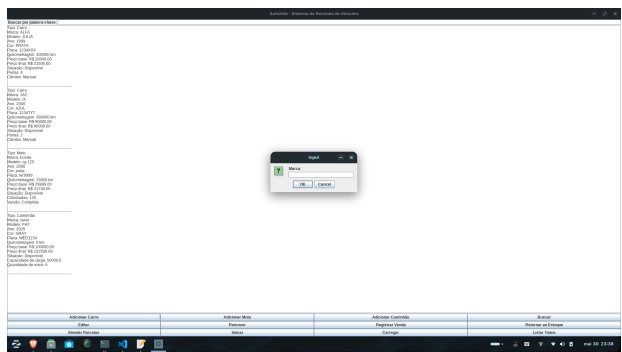


Figura 3 — Cadastro: campo marca

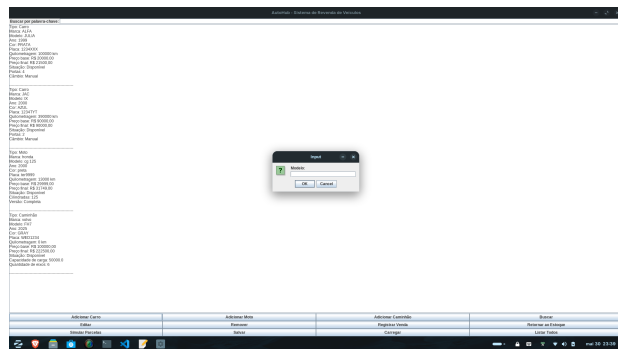


Figura 4 — Cadastro: campo modelo

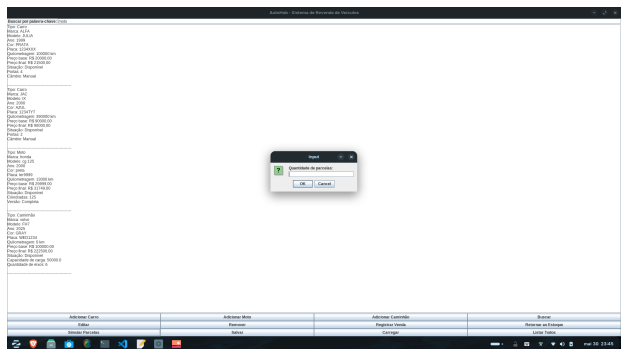


Figura 5 — Simulação de parcelas

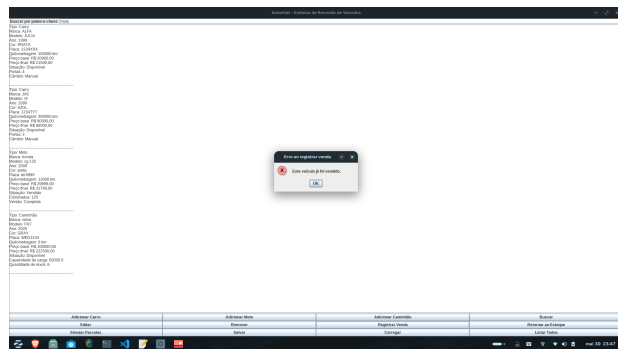


Figura 6 — Exceção: veículo já vendido